Einführung in das LSP

From Scratch mit Go und Tree-sitter

Was macht ein Language Server?

Sprachserver bieten eine Reihe von “language intelligence tools“,

darunter:

- Code completion
- Hover Informationen
- Go to definition
- ...

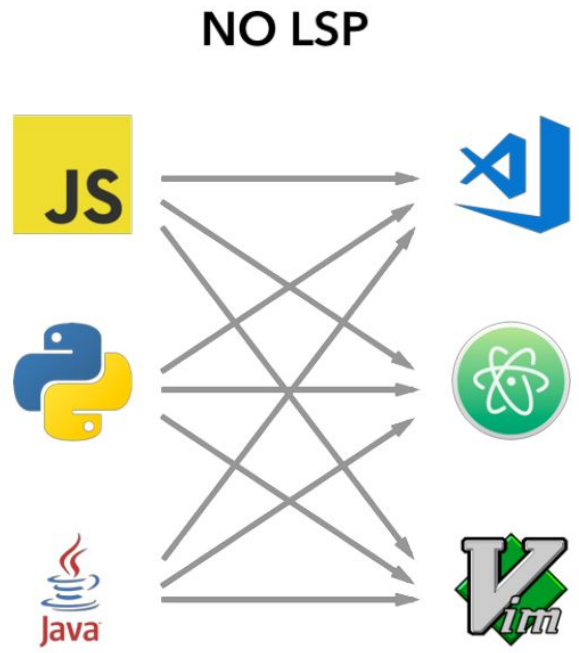
Der Editor fungiert als Client und sendet Anfragen an den Server

Vor LSP

In der Vergangenheit benötigte jeder Editor für jede Sprache ein individuelles Plugin

- Vim Python Plugin
- VS Code Go Extension
- Sublime Ruby Package
- ....

Folge: Doppelter Aufwand und inkonsistente (Feature-) Implementierungen



Die Lösung: LSP Standard

Editor (Client)

Weiß nichts über Code-analyse, dient nur als UI. Sendet Events als Folge von Benutzeraktionen

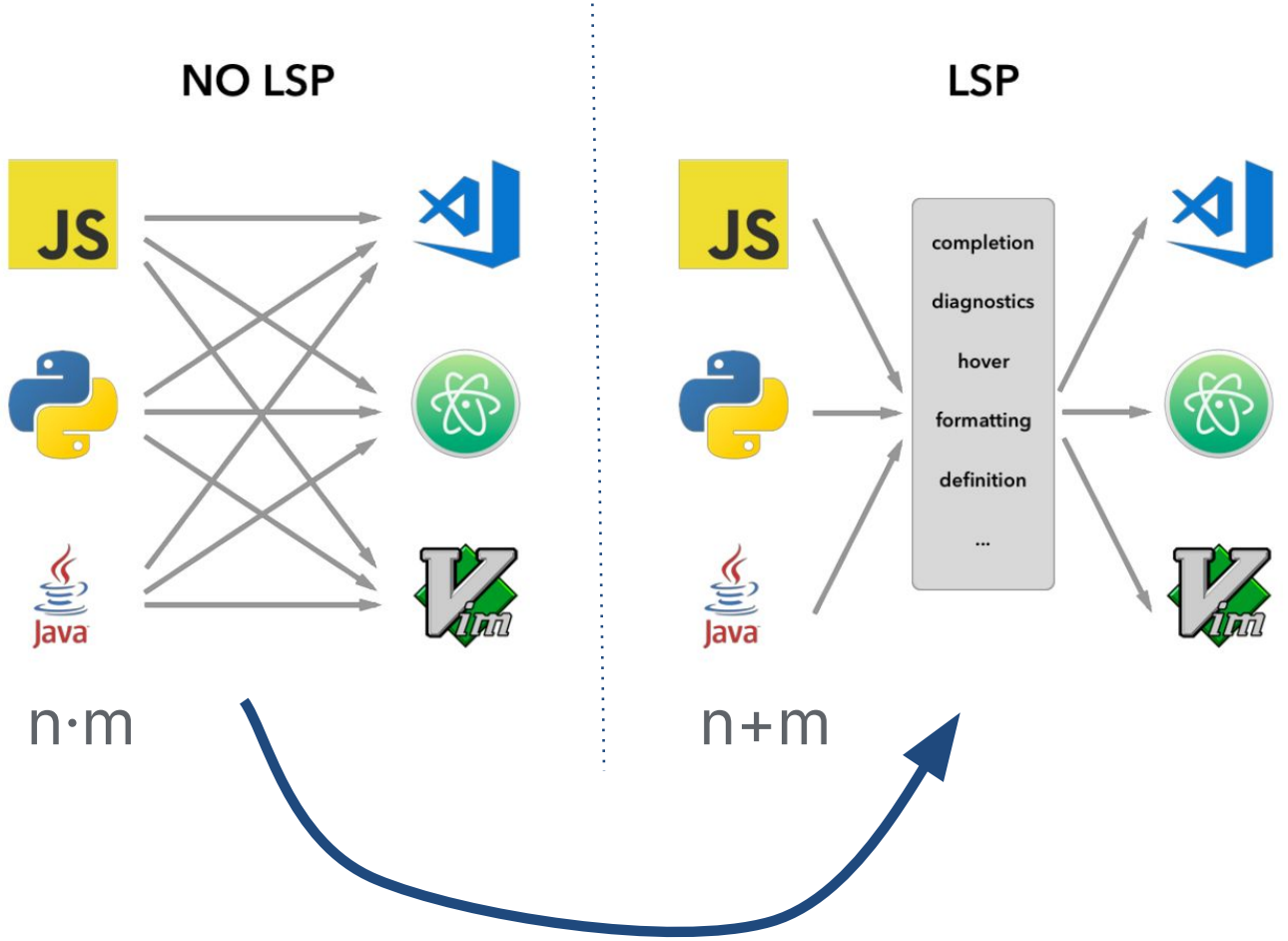
JSON-RPC

Standardisierter Nachrichtenaustausch zwischen Client und Server, basierend auf JSON-RPC v2.0

Language Server

Führt Logik aus, kennt die Syntax, findet Fehler und antwortet dem Client

Das Resultat



Der Stack

Go (Golang)

Language Server

Modern, einfach, performant

Tree-sitter

Parser, Queries

Parsed den Code zu einem Concrete Syntax Tree (CST) innerhalb von Millisekunden. Bietet eine eigene Querysprache um Codeinhalte zu extrahieren

Proof-of-Concept: "FlowLang"

Eine einfache Sprache zur Definition von Datenverarbeitungs-Pipelines.

Sie definiert, woher die Daten stammen (Quelle):

```
source “raw_user_data“ {
    path: “/data/users.csv“
}
```

welche Transformationen (Task) darauf angewendet werden:

```
task „filter_active_users“ {
    input: “raw_user_data“
    transformer: “status == ‘active‘“
}
```

und wohin sie fließen (Sink):

```
sink „active_user_report“ {
    input: “filter_active_users“
    path: “/active_users.json“
}
```

Der Server operiert nicht direkt auf dem Code, sondern auf dem von Tree-Sitter erzeugten Concrete Syntax Tree (CST)

Jedes Element im Code ist eindeutig einem Node zugeordnet, welche per Query gelesen und verändert werden kann.

CST

```
(file [0, 0] - [19, 0]
  (comment [0, 0] - [0, 27])
  (source_definition [1, 0] - [4, 1]
    name: (string_literal [1, 7] - [1, 22])
    (source_body [1, 23] - [4, 1]
      (source_body_path [2, 4] - [2, 27]
        path: (string_literal [2, 10] - [2, 27]))
      (key_value_pair [3, 4] - [3, 21]
        key: (identifier [3, 4] - [3, 12])
        value: (string_literal [3, 14] - [3, 21]))))
  (task_definition [7, 0] - [11, 1]
    name: (string_literal [7, 5] - [7, 27])
    (task_body [7, 28] - [11, 1]
      ...
      ...))
  (sink_definition [14, 0] - [18, 1]
    name: (string_literal [14, 5] - [14, 25])
    (sink_body [14, 26] - [18, 1]
      ...
        path: (string_literal [17, 10] - [17, 38]))))
```

Code

```
# Definiert die Datenquelle
source "raw_user_data" {
    path: "/data/users.csv"
    encoding: "utf-8"
}

task "filter_active_useras" {
    ...

}

sink "active_user_report" {
    ...
    path:
"/reports/active_users.json"
}
```

CST

```
file [0, 0] - [19, 0]
  (comment [0, 0] - [0, 27])
  (source_definition [1, 0] - [4, 1]
    name: (string_literal [1, 7] - [1, 22])
    (source_body [1, 23] - [4, 1]
      (source_body_path [2, 4] - [2, 27]
        path: (string_literal [2, 10] - [2, 27]))
      (key_value_pair [3, 4] - [3, 21]
        key: (identifier [3, 4] - [3, 12])
        value: (string_literal [3, 14] - [3, 21]))))
  (task_definition [7, 0] - [11, 1]
    name: (string_literal [7, 5] - [7, 27])
    (task_body [7, 28] - [11, 1]
      ...
      ...))
  (sink_definition [14, 0] - [18, 1]
    name: (string_literal [14, 5] - [14, 25])
    (sink_body [14, 26] - [18, 1]
      ...
        path: (string_literal [17, 10] - [17, 38]))))
```

Code

```
# Definiert die Datenquelle
source "raw_user_data" {
    path: "/data/users.csv"
    encoding: "utf-8"
}

task "filter_active_useras" {
    ...

}

sink "active_user_report" {
    ...
    path:
"/reports/active_users.json"
}
```

Query

```
(source_definition name: (string_literal) )
```

```
“raw_user_data”
```

Feature 1: Hover

1. Trigger

Hover über einem Textelement ausgelöst

2. Node-Typ ermitteln

Feststellen über welcher Art Node gehovert wurde

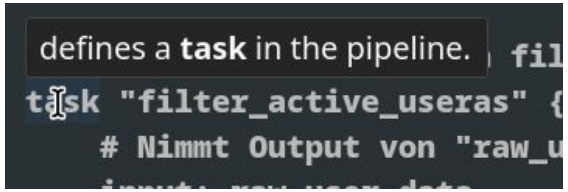
```
switch node.Kind() {
  case "source_definition":
    hoverText = "defines a source."
  case "task_definition":
    hoverText = "defines a task in the pipeline."
...
...
```

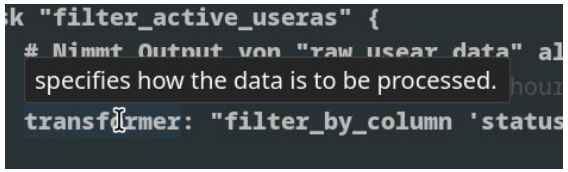
Request


```
{
  "id": 2,
  "params": {
    "textDocument": {
      "uri": "file:///proj/lsp-introduction/examples/demo.flow"
    },
    "position": {
      "character": 0,
      "line": 1
    }
  }
}
```


Response


```
{
  "id": 2,
  "result": {
    "contents": {
      "kind": "markdown",
      "value": "defines a **source**."
    }
  }
}
```



Feature 2: Diagnostics (Referenzen)

1. Trigger

Datei geöffnet (didOpen) oder bearbeitet (didChange)

2. Definitionen sammeln

Alle definierten Sources, Tasks und Sinks ermitteln

```
var definedNames map[string]bool
raw_user_data -> true
filter_active_users -> true
...
```

3. Referenzen gegenprüfen

Prüfen ob alle referenzierten Objekte definiert sind

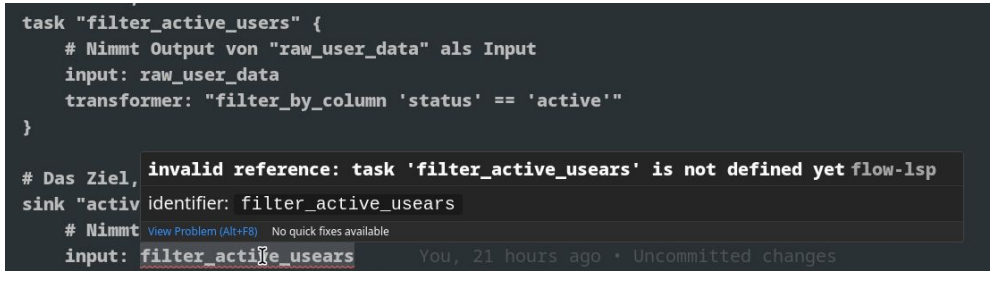
```
if !definedNames[refName] {
    ...
```

```
{
  "textDocument":
"file:///proj/lsp-introduction/examples/demo.flow",
  "contentChanges": [
    {
      "text": "
      # Definiert die Datenquelle
      source "raw_user_data" {
        path: "/data/users.csv"
        encoding: "utf-8"
      }
      # Ein Task, der die Daten filtert
      task "filter_active_users" {
        # Nimmt Output von "raw_user_data" als Input
        input: raw_user_data
        transformer: "filter_by_column 'status' == 'active'"
      }
      # Das Ziel, wo die finalen Daten gespeichert werden
      sink "active_user_report" {
        # Nimmt den Output des Anonymisierungs-Tasks
        input: filter_active_usears
        path: "/reports/active_users.json"
      }"
    }
  ]
}
```


Request


```
{
  "method": "textDocument/publishDiagnostics",
  "params": {
    "uri": "file:///proj/lsp-introduction/examples/demo.flow",
    "diagnostics": [
      {
        "range": {
          "start": {
            "line": 16,
            "character": 11
          },
          "end": {
            "line": 16,
            "character": 31
          }
        },
        "severity": 1,
        "source": "flow-lsp",
        "message": "invalid reference: task 'filter_active_usears' is
                    not defined yet"
      }
    ]
  }
}
```


Response



Vielen Dank